CONSULTA RÁPIDA · AF

Cola para a prova

Todas as linhas dos modelos estão comentadas: o comentário fica logo acima da linha que ele explica. A AF é **com consulta** aos materiais (e sem IA). Deixe esta página aberta: tem os includes, as assinaturas, os modelos prontos para adaptar e os erros que mais tiram ponto.

Compilar, rodar e testar

```
# ---------- COMPILAR ----------
# -Wall: mostra todos os avisos do compilador
# -g: inclui informações de debug (necessário para gdb e valgrind apontarem linhas)
# -o q1: nome do executável gerado
# -pthread: obrigatório para threads e semáforos
gcc -Wall -g q1.c -o q1 -pthread
# processos, arquivos e sinais não precisam de -pthread
gcc -Wall -g q2.c -o q2

# ---------- MEMÓRIA ----------
# roda o programa sob o valgrind; o objetivo é "ERROR SUMMARY: 0 errors" e "All heap
# blocks were freed"
valgrind --leak-check=full ./q4 2 arq.txt

# ---------- SINAIS ----------
# roda em background (o & devolve o terminal) e mostra o pid do processo
./q3 &
# envia SIGINT (o mesmo que o Ctrl+C) ao processo <pid>
kill -s SIGINT <pid>
# envia SIGUSR1
kill -USR1 <pid>
# envia SIGKILL (9): mata na hora, não pode ser capturado
kill -9 <pid>
# código de saída do último comando (130 = morreu por SIGINT, 128 + 2)
echo $?
# traduz o código de saída no nome do sinal (ex.: INT)
kill -l $?

# ---------- TEMPO E REPETIÇÃO ----------
# mede o tempo total (serve para conferir o limite de threads simultâneas)
time ./q4 2 arq.txt
# roda 20 vezes, cada execução numa linha só, para ver as ordens possíveis
for i in $(seq 20); do ./q1 | tr -d '\n'; echo; done

# ---------- MANUAIS (em caso de dúvida, use o man) ----------
# seção 2 = chamadas de sistema
man 2 fork
# wait, waitpid e as macros WIFEXITED etc.
man 2 wait
# seção 3 = funções da biblioteca
man 3 execvp
# flags O_* e mode
man 2 open
# o que read retorna (0 = EOF)
man 2 read
```

```
# lista de sinais e seus números
man 7 signal
# struct sigaction
man 2 sigaction
# visão geral dos semáforos POSIX
man sem_overview
# pthread_create e seus argumentos
man 3 pthread_create
```

Includes por assunto

ASSUNTO	HEADERS	FUNÇÕES
Básico	`stdio.h stdlib.h string.h`	`printf snprintf malloc realloc free atoi atof atol strtol exit strlen strcmp strsignal`
Processos	`unistd.h sys/types.h sys/wait.h`	`fork getpid getppid execvp wait waitpid sleep` · macros `WIFEXITED WEXITSTATUS WIFSIGNALED WTERMSIG WNOHANG`
Arquivos	`fcntl.h unistd.h sys/stat.h errno.h`	`open read write close lseek perror` · `O_RDONLY O_WRONLY O_RDWR O_CREAT O_TRUNC O_APPEND O_EXCL`
Sinais	`signal.h`	`sigaction sigemptyset sigaddset kill raise` · `SIG_DFL SIG_IGN`
Threads	`pthread.h`	`pthread_create pthread_join pthread_mutex_lock pthread_mutex_unlock PTHREAD_MUTEX_INITIALIZER`
Semáforos	`semaphore.h`	`sem_init sem_wait sem_post sem_destroy`
Tempo/aleatório	`time.h`	`time srand rand random`

Modelo: fork + execvp + waitpid (com retry)

```c
#include <stdio.h>        // printf, perror
#include <stdlib.h>       // exit
#include <string.h>       // strsignal (nome legível do sinal)
#include <unistd.h>       // fork, execvp
#include <sys/types.h>    // tipo pid_t
#include <sys/wait.h>     // waitpid e as macros WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG

// argc = quantos argumentos; argv = os argumentos (strings)
int main(int argc, char *argv[]) {
    // esperamos o nome do programa + 2 argumentos
    if (argc != 3) {
        // argv[0] é o nome do próprio programa
        printf("USO: %s <a> <b>\n", argv[0]);
        // sai indicando erro
        return 1;
    }
    // começa > 0 para entrar no laço pela primeira vez
    int ret = 1;
    // repete enquanto houver falha temporária (retorno > 0)
    while (ret > 0) {
        // clona o processo: retorna 0 no filho e o pid do filho no pai
        pid_t filho = fork();
        // ---- só o FILHO entra aqui ----
```

```
        if (filho == 0) {
            // argv do novo programa: nome primeiro, NULL no fim
            char *args[] = {"./programa", argv[1], argv[2], NULL};
            // troca este processo pelo ./programa (não retorna se der certo)
            execvp(args[0], args);
            // só chega aqui se o exec FALHOU: mostra o motivo
            perror("execvp");
            // encerra o filho para ele não continuar o laço
            exit(1);
        }
        // ---- PAI: variável que recebe o status de término ----
        int wst;
        // bloqueia até ESSE filho terminar (0 = sem opções)
        waitpid(filho, &wst, 0);
        // o filho terminou normalmente (return/exit)?
        if (WIFEXITED(wst)) {
            // pega o valor de retorno; o (char) transforma 255 de volta em -1
            ret = (char) WEXITSTATUS(wst);
            // mostra o retorno
            printf("retornou %d\n", ret);
        // ou o filho foi morto por um sinal?
        } else if (WIFSIGNALED(wst)) {
            // número e nome do sinal
            printf("morreu com sinal %d (%s)\n", WTERMSIG(wst), strsignal(WTERMSIG(wst)));
            // trata como falha e tenta de novo
            ret = 1;
        }
    // sai do laço com ret == 0 (sucesso) ou ret < 0 (erro de parâmetro)
    }
    // fim do pai
    return 0;
}
```

Modelo: handler que salva em arquivo e termina pelo sinal

```
#include <stdio.h>        // printf, snprintf, perror
#include <stdlib.h>       // exit
#include <unistd.h>       // getpid, write, close, usleep
#include <signal.h>       // sigaction, sigemptyset, sigaddset, raise, SIG_DFL
#include <fcntl.h>        // open e as flags O_WRONLY, O_CREAT, O_TRUNC

// GLOBAL: o handler só enxerga variáveis globais
long estado = 0;

// grava "estado" em estado.txt, como texto
void salva(void) {
    // abre para escrita, cria se não existir, apaga o conteúdo antigo; 0644 =
    // rw-r--r--
    int fd = open("estado.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    // open retorna -1 em erro: mostra o motivo e desiste
    if (fd == -1) { perror("open"); return; }
    // buffer para o número convertido em texto
    char buf[64];
    // converte o long em texto; n = quantidade de caracteres gerados
    int n = snprintf(buf, sizeof(buf), "%ld\n", estado);
    // escreve exatamente n bytes no arquivo
    write(fd, buf, n);
    // fecha o arquivo (libera o fd)
    close(fd);
```

```c
}

// handler: assinatura obrigatória void f(int); sig = número do sinal recebido
void handler(int sig) {
    // salva o estado no arquivo
    salva();
    // avisa na tela
    printf("salvo: %ld\n", estado);
    // nova configuração para o SIGINT
    struct sigaction d;
    // SIG_DFL = comportamento padrão (terminar o processo)
    d.sa_handler = SIG_DFL;
    // sem opções especiais
    d.sa_flags = 0;
    // nenhum sinal extra bloqueado
    sigemptyset(&d.sa_mask);
    // aplica: o próximo SIGINT vai matar o processo
    sigaction(SIGINT, &d, NULL);
    // envia SIGINT para si mesmo e morre "de verdade" pelo sinal ($? = 130)
    raise(SIGINT);
}

int main(void) {
    // mostra o pid para podermos usar kill de outro terminal
    printf("PID: %d\n", getpid());
    // configuração do handler
    struct sigaction s;
    // função que vai rodar quando o sinal chegar
    s.sa_handler = handler;
    // sem opções especiais
    s.sa_flags = 0;
    // começa com a máscara vazia (é obrigatório inicializar)
    sigemptyset(&s.sa_mask);
    // opcional: bloqueia SIGTERM enquanto o handler roda
    sigaddset(&s.sa_mask, SIGTERM);
    // registra o handler para SIGINT (registre ANTES do laço)
    sigaction(SIGINT, &s, NULL);
    // laço infinito simulando trabalho
    while (1) {
        // atualiza o estado
        estado++;
        // dorme 1 ms (1000 microssegundos)
        usleep(1000);
    }
}
```

Modelo: read_line com read (tamanho arbitrário)

```c
// devolve a próxima linha de fd (sem o \n) ou NULL no fim do arquivo
char *read_line(int fd) {
    // capacidade atual do buffer (bytes alocados)
    int cap = 16;
    // quantos caracteres já foram guardados
    int len = 0;
    // retorno do read: 1 = leu um byte, 0 = EOF, -1 = erro
    int lidos;
    // aloca o buffer inicial no heap
    char *buf = malloc(cap);
    // guarda o byte lido
```

```
    char c;
    // lê 1 byte por vez enquanto houver dados
    while ((lidos = read(fd, &c, 1)) > 0) {
        // fim da linha: para (o \n não é guardado)
        if (c == '\n') break;
        // sem espaço para mais um char + o '\0'?
        if (len + 1 >= cap) {
            // dobra a capacidade
            cap *= 2;
            // realoca: aceita linhas de qualquer tamanho
            buf = realloc(buf, cap);
        }
        // guarda o char e avança a posição
        buf[len++] = c;
    }
    // chegou ao EOF (ou erro) sem ler nada: não há mais linhas
    if (lidos <= 0 && len == 0) {
        // libera o buffer não usado (o valgrind cobra)
        free(buf);
        // avisa quem chamou que o arquivo acabou
        return NULL;
    }
    // termina a string (read não coloca o '\0')
    buf[len] = '\0';
    // devolve a linha; quem chamou deve dar free()
    return buf;
}
```

Modelo: N threads, struct por thread, limite por semáforo e mutex

```
#include <stdio.h>        // printf
#include <stdlib.h>       // malloc, free, atoi
#include <pthread.h>      // pthread_create, pthread_join, mutex
#include <semaphore.h>    // sem_t, sem_init, sem_wait, sem_post, sem_destroy

// tudo o que cada thread recebe (um bloco por thread, no heap)
typedef struct {
    // número da thread
    int id;
    // dado de entrada
    int entrada;
    // saída: a thread escreve o resultado aqui
    int res;
    // ponteiro para o semáforo compartilhado (nunca uma cópia!)
    sem_t *limite;
    // ponteiro para o mutex compartilhado
    pthread_mutex_t *mutex;
} t_arg;

// variável global compartilhada (precisa de mutex)
long total = 0;

// assinatura obrigatória de thread: void *f(void *)
void *tarefa(void *_arg) {
    // converte o void* para o tipo real do argumento
    t_arg *a = _arg;
    // pede uma "vaga": bloqueia se já há K threads trabalhando
    sem_wait(a->limite);
    // o trabalho em si (resultado vai para a struct)
```

```c
    a->res = a->entrada * 2;
    // devolve a vaga para outra thread
    sem_post(a->limite);
    // entra na região crítica (só uma thread por vez)
    pthread_mutex_lock(a->mutex);
    // lê, soma e escreve a global com segurança
    total += a->res;
    // sai da região crítica
    pthread_mutex_unlock(a->mutex);
    // nada a retornar (o resultado está na struct)
    return NULL;
}

int main(int argc, char *argv[]) {
    // quantas threads criar
    int n = 10;
    // máximo simultâneo: vem do 1º argumento ou é 2 por padrão
    int k = (argc > 1) ? atoi(argv[1]) : 2;
    // semáforo que conta as vagas
    sem_t limite;
    // 0 = entre threads do mesmo processo; valor inicial = k vagas
    sem_init(&limite, 0, k);
    // cria o mutex já inicializado (destravado)
    pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
    // vetor com os IDs das threads
    pthread_t *tids = malloc(sizeof(pthread_t) * n);
    // vetor de ponteiros para os argumentos
    t_arg **args = malloc(sizeof(t_arg *) * n);
    // cria as n threads
    for (int i = 0; i < n; i++) {
        // argumento PRÓPRIO de cada thread, no heap
        args[i] = malloc(sizeof(t_arg));
        // preenche o id
        args[i]->id = i;
        // preenche a entrada
        args[i]->entrada = i;
        // todas apontam para o MESMO semáforo
        args[i]->limite = &limite;
        // todas apontam para o MESMO mutex
        args[i]->mutex = &mutex;
        // cria a thread: guarda o id, opções padrão, função e argumento
        pthread_create(&tids[i], NULL, tarefa, args[i]);
    }
    // espera as threads na ordem de criação
    for (int i = 0; i < n; i++) {
        // bloqueia até a thread i terminar
        pthread_join(tids[i], NULL);
        // agora é seguro ler o resultado
        printf("THREAD %02d RES %d\n", args[i]->id, args[i]->res);
        // libera o argumento DEPOIS do join
        free(args[i]);
    }
    // todas terminaram: o total está completo
    printf("TOTAL: %ld\n", total);
    // libera o vetor de ponteiros
    free(args);
    // libera o vetor de IDs
    free(tids);
    // destrói o semáforo
    sem_destroy(&limite);
```

```
    return 0;
}
```

Modelo: diagrama de dependências

```c
#include <stdio.h>        // printf
#include <pthread.h>      // pthread_create, pthread_join
#include <semaphore.h>    // sem_t e funções de semáforo

// Diagrama deste exemplo: A -> C, A -> D, B -> C  ("C depende de A e de B", "D depende de A")
// um semáforo por parte que tem setas SAINDO (A e B); todos começam em 0
sem_t sA, sB;

void *t1(void *a) {
    // executa a parte A
    printf("A\n");
    // avisa "A terminou" para C...
    sem_post(&sA);
    // ...e de novo para D (um post para cada seta que sai de A)
    sem_post(&sA);
    return NULL;
}
void *t2(void *a) {
    // executa a parte B
    printf("B\n");
    // avisa "B terminou" (só C depende de B: um post)
    sem_post(&sB);
    return NULL;
}
void *t3(void *a) {
    // C espera A (consome um dos dois posts de sA)
    sem_wait(&sA);
    // C espera B
    sem_wait(&sB);
    // só imprime depois de A e B
    printf("C\n");
    return NULL;
}
void *t4(void *a) {
    // D espera A (consome o outro post de sA)
    sem_wait(&sA);
    // só imprime depois de A
    printf("D\n");
    return NULL;
}

int main(void) {
    // valor inicial 0 = "A ainda não aconteceu"
    sem_init(&sA, 0, 0);
    // valor inicial 0 = "B ainda não aconteceu"
    sem_init(&sB, 0, 0);
    // vetor de ponteiros para as funções (para criar em laço)
    void *(*f[4])(void *) = {t1, t2, t3, t4};
    // IDs das threads
    pthread_t tid[4];
    // cria TODAS as threads em laço
    for (int i = 0; i < 4; i++)
        // a thread i executa f[i]; sem argumento (os semáforos são globais)
        pthread_create(&tid[i], NULL, f[i], NULL);
```

```
    // espera TODAS (só no fim, para não criar dependências extras)
    for (int i = 0; i < 4; i++)
        pthread_join(tid[i], NULL);
    // libera os semáforos
    sem_destroy(&sA);
    sem_destroy(&sB);
    return 0;
}
```

Modelo: barreira para N threads

```
#include <stdio.h>        // printf
#include <pthread.h>      // threads e mutex
#include <semaphore.h>    // semáforo
#define N 4               // número de threads que precisam se encontrar na barreira

// quantas threads já chegaram (compartilhado: precisa de mutex)
int chegaram = 0;
// protege o contador "chegaram"
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
// semáforo onde as threads ficam presas (começa em 0)
sem_t barreira;

void *tarefa(void *a) {
    // trabalho antes do ponto de encontro
    printf("Antes da barreira\n");
    // trava para mexer no contador
    pthread_mutex_lock(&m);
    // registra que esta thread chegou
    chegaram++;
    // 1 se esta for a última a chegar
    int ultimo = (chegaram == N);
    // destrava (nunca espere no semáforo segurando o mutex!)
    pthread_mutex_unlock(&m);
    // a última thread...
    if (ultimo) {
        // ...faz N-1 posts:
        for (int i = 0; i < N - 1; i++)
            // cada post libera exatamente uma thread presa no sem_wait
            sem_post(&barreira);
    // as outras N-1 threads...
    } else {
        // ...ficam esperando a última chegar
        sem_wait(&barreira);
    }
    // só roda depois que TODAS chegaram
    printf("Depois da barreira\n");
    return NULL;
}

int main(void) {
    // 0 = ninguém pode passar ainda
    sem_init(&barreira, 0, 0);
    // IDs das threads
    pthread_t t[N];
    // cria as N threads, todas com a mesma função
    for (int i = 0; i < N; i++)
        pthread_create(&t[i], NULL, tarefa, NULL);
    // espera todas terminarem
```

```
    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);
    // libera o semáforo
    sem_destroy(&barreira);
    return 0;
}
```

Erros que mais tiram ponto

PROCESSOS

- Esquecer o `NULL` no fim do vetor do `execvp`, ou esquecer o `argv[0]`.
- Código depois do `execvp` sem `exit`: se o exec falhar, o filho continua o laço e cria mais processos.
- Ler `WEXITSTATUS` sem cast `(char)`, de modo que -1 vira 255.
- Usar `system()` quando é proibido.
- Não esperar os filhos (deixa zumbis e o pai sai antes).

ARQUIVOS

- Usar `fopen` / `fprintf` / `fgets` / `getline` quando o enunciado pede só `open` / `read` / `write`.
- Esquecer o `O_CREAT` ou o mode (`0644`), ou o `O_TRUNC` (o arquivo fica com restos do conteúdo anterior).
- Escrever `sizeof(buf)` bytes em vez de `n`, o que grava lixo.
- Esquecer o `'\0'` depois do `read` antes de usar como string.

SINAIS

- Registrar o handler depois do laço infinito: ele nunca é registrado.
- Assinatura do handler diferente de `void h(int)`.
- Esquecer o `sigemptyset(&s.sa_mask)` e o `s.sa_flags = 0` (lixo na struct).
- O dado que o handler precisa não está numa global.

THREADS E SEMÁFOROS

- Compilar sem `-pthread`.
- Passar `&i` do laço ou o endereço de uma local de função que já retornou.
- Não fazer join em todas: o main termina e mata as threads.
- Ler o resultado antes do join.
- `free` antes do join, ou não liberar nada (o valgrind acusa).
- Usar `join` ou `sleep` para impor ordem (cria dependências extras, e a questão vale no máximo 50%).
- Semáforo de dependência iniciado em 1, ou poucos `post` para muitos dependentes, levando a deadlock.
- `sem_wait` com o mutex travado, levando a deadlock.
- Mudar ou adicionar `printf` quando o enunciado proíbe, ou imprimir fora do formato exato (espaços e `\n` extras).

Estratégia para as 3 horas

1. Leia o README inteiro e anote as **rubricas em %**: elas dizem o que priorizar.
2. Comece pela questão que você domina mais. Mire primeiro nos itens de maior peso.
3. **Entregue a cada questão terminada**: o Blackboard aceita várias tentativas. Rezipe a pasta inteira e não renomeie os arquivos.
4. Compile sempre com `-Wall` e teste com os programas de referência (`q2_ref`) quando houver.
5. No fim, rode o valgrind e confira o formato exato da saída.